

Active Learning Methodology for Advanced Agile Software Engineering and DevOps Practices

Juan Ignacio Irabedra
Universidad ORT Uruguay
Montevideo, Uruguay
juanignacioirabedra@gmail.com

Martin Solari
Universidad ORT Uruguay
Montevideo, Uruguay
martin.solari@ort.edu.uy

Abstract—Creation of digital products and services requires today high-performance interdisciplinary teams that respond quickly to customer needs. This context presents a great challenge for software engineering education, where it seeks to give students an active learning environment of professionally relevant and technologically updated skills. This report presents a classroom experience with a focus on active learning pedagogical strategies for advanced agile software engineering and DevOps practices. The instructional design of the course was based on two main pedagogical methodologies: Team-Based Learning and Project-Based Learning. The course was developed on digital platforms that support both the pedagogical approach and software engineering teamwork and software quality practices. Evaluation was carried out from both teacher's and student's perspectives, using quantitative and qualitative analysis. Lessons learned from the experience are presented to enable adoption of these active learning methodologies in other software engineering courses.

Keywords—software engineering, active learning, Team-Based Learning, Project-Based Learning, digital education

I. INTRODUCTION

Software Engineering (SE) education has undergone a major transformation in recent decades. This transformation has three main influences: the current professional practice of the discipline, the change to a competency-oriented educational system and the use of digital learning platforms.

Despite its youth as a discipline, SE has undergone several paradigm shifts. Initially based on traditional engineering methods, its most relevant transformation is the adoption of agile methodologies [1]. In parallel, SE has adapted to emerging computational paradigms and new technologies (mobile, cloud computing, microservices, DevOps) that impact the nature of the digital product/service and the teams that develop it [2]. Today, software is expected to be deployed continuously, operated and maintained by interdisciplinary teams. To apply opportunities that deliver value to customers, high-performance teams have a high degree of methodological autonomy, along with criteria for alignment with business objectives.

In another axis of changes, engineering education has set its focus on meaningful learning of competencies in practical professional contexts. University curricula does not only emphasize theoretical content and memorization, but also learning relevant skills to solve problems effectively. These skills are in high demand from the industry and in the SE area it requires combining technical skills with human and team skills [3], [4].

University education has reacted to this change by changing curricula, seeking competencies orientation and transversal learning of skills in different knowledge areas. On the other hand, the advancement of instructional methods has made it possible to incorporate active learning strategies that

promote student engagement and deep learning of skills in realistic contexts [5], [6].

Like all areas of human activity, education has also undergone a transformation related to the use of digital technology. In addition to facilitating access to content through digital media (including audiovisual content and software artifacts), digital technology allows synchronous or asynchronous online course activities. After a forced incorporation due to the COVID-19 pandemic, universities have stabilized the use of digital tools and platforms for online or hybrid courses [7], [8]. Learning Management Systems are commonly used to support online courses. Pedagogy-focused digital platforms that enable active learning strategies and hybrid teamwork have recently emerged.

This article presents a classroom experience report on incorporating active learning methodologies into an advanced agile SE course using digital platforms. The pedagogical strategies deployed in the course pursue three objectives simultaneously:

- Teaching advanced agile SE practices for high-performance interdisciplinary teams, in the context of a DevOps culture.
- Enable meaningful learning of professionally relevant skills (both technical and soft skills) and promote student engagement, through active learning strategies.
- Effective use of digital learning platforms, adapted to the pedagogical methodology and technologically updated for SE teamwork and software quality activities.

The educational intervention was evaluated from the perspective of students and teachers using quantitative and qualitative approaches. To enable the adoption of these active learning strategies in other SE teaching contexts, lessons learned and lines of future evolution of the classroom experience are presented.

The rest of the article is structured as follows. Section II presents a brief review of previous work on active learning methodologies in SE. Section III presents the historical evolution of SE and how it influences the advanced SE and DevOps course learning objectives. Section IV presents its instructional design, applying different pedagogical strategies in the course units. Section V focuses on digital platforms that support pedagogical strategies, as well as teamwork and software quality practices. Section VI presents the pedagogical evaluation and lessons learned from the classroom experience. Finally, conclusions are presented in section VII.

II. ACTIVE LEARNING METHODOLOGIES IN SE

Active learning is defined as a set of strategies that promote greater student involvement in the educational process, with the aim of promoting deeper learning of concepts and skills. Active learning can be contrasted with

traditional teaching methods, where the student is a mere recipient of the knowledge imparted by the teacher through lessons. Active learning strategies usually imply 'learning by doing', application of knowledge in realistic contexts and high-order cognitive strategies, such as reflection and meta-learning [9]. There are multiple proposals for active learning methodologies and dynamics. Some examples are Games, Project and Problem-Based Learning, Team-Based Learning, Think-Pair-Share [5], [10]. Different higher education institutions have established pedagogical innovation centers with initiatives to disseminate active learning strategies.

The incorporation of these strategies has also been a topic of recent educational research in the area of engineering [9], [11] and in the area of SE in particular [12], [13]. The main results of the case studies of teaching experiences and secondary studies show that project-based active learning strategies [14], [15] and game-based [16], [17] are the most common active learning strategies. Educational research shows the positive impacts of active learning both from the student's and from the teacher's point of view [11], [18], [19], [20].

III. SE EVOLUTION AND COURSE LEARNING OBJECTIVES

The seminal proposal of the Agile Manifesto emphasizes the fundamental values of this paradigm: focus on people and teams, adapt to changes and deliver value continuously. In the first wave of agility, methodological development and adoption of concrete proposals, such as XP and particularly Scrum, were observed. Later, technology companies and open-source communities transitioned to more flexible models that allowed teams to experiment and generate improvements in a bottom-up way. In many cases, methods from Japanese production culture such as Lean and Kanban were incorporated, but also in combination with traditional project management methods and models to scale agility.

Nowadays, SE is practiced in interdisciplinary teams with a high degree of autonomy and criteria to align with business objectives. Teams work within the framework of a value stream and are responsible for developing, operating, and maintaining a particular digital service on an ongoing basis. Larger-scale organizations use microservices-based software architectures that allow teams to autonomously make decisions about methodology and technology.

DevOps is a cultural movement that promotes collaboration between different areas for the development and integrated operation of software-based services. It emphasizes collaboration and communication between teams with a shared view of quality goals. DevOps practices are largely supported by the ability to continuously deploy and deliver value enabled by cloud architectures, which also implies expanded challenges of scalability, security, testability, and continuous business feedback.

The Agile Software Engineering 2 course's goal is to learn advanced concepts of agility in high-performance team contexts. Emphasis is placed on the application of concepts on quality, maintenance, software evolution and the relationship between development and operations activities (DevOps), in the context of the business goals. Several practices are applied to reduce delivery cycles, contribute to the value stream and the implementation of solutions with a global reach. In turn, the main supporting technologies for these processes are introduced throughout the software life cycle.

To define the desired outcomes of learning in the course, we have structured learning objectives according to the levels of Bloom's taxonomy, using the verbs 'know' and 'apply' associated with each objective [21]. Although these verbs focus on the lower levels of the revised Bloom taxonomy, the course proposes skill-building activities such as research, teamwork, and continuous learning that also require higher-order thinking levels (analyze, evaluate, create).

- *Knowing* the principles of value delivery and the views on quality established by the stakeholders of a software project.
- *Knowing* the interaction between the different areas involved in the development and operation of software-based products or services.
- *Knowing* the patterns that favor the delivery of value.
- *Applying* practices and technologies in a deployment pipeline that reduces the time between a modification to the code and the software reaching the deliverable state.
- *Applying* SCM, construction, and deployment techniques and tools in a coordinated manner to automate deployment pipeline tasks.
- *Applying* quality assurance and testing strategies integrated into the deployment pipeline. Implement different levels of testing using test automation technologies.
- *Knowing* the patterns for continuous learning and experimentation and team structures at the organizational level.

The course is composed of five units: 1) Introduction to DevOps, 2) Continuous Delivery Practices, 3) Testing Automation 4) Telemetry and Feedback, 5) Continuous Learning and Experimentation.

In the instructional design of a course based on flipped classes, an appropriate selection of study material is essential. Given that the subject matter requires a systemic understanding of complex issues and a contemporary paradigm of SE, we have selected a main bibliographic source to give unity to this vision and facilitate the study process for the students. The DevOps Handbook discusses advanced agile SE concepts. It is structured based on the proposal of three paths for high-performance teams: flow, feedback, continuous learning and experimentation. Complementary material is also used: Accelerate books, Agile Testing, Software Delivery Guide and Martin Fowler's blog and DORA project (DevOps Research and Assessment). Web references to the tech tools sites (GitHub Actions, Cucumber, and Cypress) were also included. We consider that, in the flipped classroom methodology, the appropriate choice of sources is a fundamental component for the learning of software engineering in a context of up-to-date practices supported by evidence.

IV. INSTRUCTIONAL COURSE DESIGN

The pedagogical design of the course is based on different active learning strategies. Team-Based Learning (TBL) and Project-Based Learning (PBL) are the main methodologies for the course pedagogy.

TBL is a pedagogical methodology for active, team-based learning [22]. The method proposes a sequence of learning

QP1

QP2

activities that is repeated for each thematic unit: preparation, individual and team readiness assessment, clarifications and appeals, application exercises. The methodology balances individual contribution and teamwork, preparation study outside the classroom, and application in exercises. It also incorporates peer evaluation and feedback to strengthen teamwork. These aspects of the methodology contribute to the course objectives on learning team skills and continuous learning of technologies for software development [23], [24].

In the context of application exercises, the course fosters both activities that foster high order thinking about the preparation material, as well as hands-on activities that seek to apply cutting-edge technologies that allow the teams to apply DevOps concepts in real-life scenarios. Examples of such activities are platform engineering using Kubernetes and infrastructure as code using Terraform, and how they relate to a deployment pipeline.

PBL is a pedagogical strategy extensively used in engineering education [25]. As a discipline that focuses on solving problems through the integration of technology, the complex and authentic contexts used by PBL encourage significant learning of skills. This strategy has been extensively researched in the field of software engineering education and positive results have been reported in student learning and motivation [26], [27].

In the DevOps project, teams are self-selected and are presented with the challenge of maintaining software developed by another team in a previous course. While students are familiar with the technologies, they don't know the concrete software project and must analyze it as part of the maintenance tasks. The objectives of the project include new features and bug fixes, as well as technical debt analysis, testing improvement, and a revised development process applying software quality strategies. This project is an exception to the exercises used in other courses that usually involve creating a project from scratch and in the context of limited quality objectives.

The project is developed over 11 weeks, with 4 intermediate deliveries and a final release of code and documentation. The project has broad quality objectives, students are expected to actively participate in defining specific objectives and iteratively reflect on the process using team retrospectives. Throughout the project, students must record individual contributions and have a minimum effort dedication per week to engineering activities.

Although the project is continuously developed using a Kanban board adapted by the team itself, each intermediate delivery focuses on results in specific areas:

- Definition of the management framework, engineering process, versioning.
- Analysis of technical debt according to a quality model defined by the teams.
- Implementation of a deployment pipeline.
- Issue correction (selected based on prioritization according to the quality model previously defined).
- Development of new features using BDD cycles.

- Deepening and extending QA activities as well as test automation.

In addition, teams should investigate one of the SE technologies or practices applied in the project to demonstrate it to the class and write a short article. The objective of this activity is to practice technological research and cross-team communication, a common practice in organizations that must continuously adapt to changes in technological ecosystems.

The course also includes an individual written exam that represents 30% of the total score. This is the only component of the course that uses a traditional assessment method. Table I presents the main pedagogical strategies, with each percentual weight in individual and team assessment.

TABLE I. PEDAGOGICAL STRATEGIES AND ASSESSMENT

Paragological strategies	Individual assesment	Team assesment
Team-Based Learning as the main pedagogical method.	15 %	25 %
Project-Based Learning for the development of team skills in real context, outside of classroom time.		25 %
Technology research, writing a short article and demonstration to class.		5 %
Individual written exam, covers all course topics.	30 %	

V. DIGITAL LEARNING AND SE PLATFORMS

QP4

In today's higher education, it is important to provide students and teachers with a digital learning experience. Digital platforms must go beyond the availability of asynchronous content: they must adapt to skills learning objectives and the pedagogical approach of the course.

GitHub is the leading global platform for collaborative software development. It is used by millions of professional developers and businesses around the world. Its functions go beyond source code versioning, support teamwork, and the management of the entire software lifecycle. It can be used to integrate quality assurance, project management, security, and deployment activities. Using GitHub as the primary platform in an advanced agile SE and DevOps course provides realistic context for exercising technology skills using a state-of-the-art toolchain [28]. The direct use of the platform becomes an educational asset for students who can immediately transfer this knowledge into professional practice.

We use GitHub Education¹ tools to create an organization for the manipulation of all software artifacts (exercises and DevOps project), GitHub allows not only to store source code and documentation, but also to manage projects in the platform. In the course each team uses a Kanban board in GitHub Projects, reports bugs and quality using Issues, conducts code reviews supported by Pull Requests. A particularly relevant aspect is the use of GitHub Actions workflows to build a CI/CD pipeline as a foundational pattern for continuous delivery of quality-oriented systems. The course integrates SE life-cycle technologies such advanced testing and QA tools: Behavior-Driven Development using Cucumber and functional testing with Cypress. All these tools are configured and applied by each team according to their own process decisions.

¹ <https://github.com/education>

The DevOps project entails analyzing, planning and improving technical debt. The tech stack involves ASP.NET Core and Angular. These technologies are learned in previous courses to build a fullstack web application mainly composed of CRUD requirements. The students can deepen their mastery of such tech stack. In prior courses, students focused on coding skills and making the application work. In the current subject, they must interweave the technical skills previously obtained within a collaborative management and engineering framework.

Each team must complete and submit all exercises in a GitHub repository. From the teacher's perspective, we use GitHub Classroom for creating assignments for each unit and repositories for the DevOps projects. This allows the student to have access to all the features of the GitHub platform, while facilitating administration for the teacher.

From a pedagogical point of view, we have adopted the InteDashboard² platform that manages the entire TBL methodology in a digital platform [29]. At the beginning of each thematic unit, students take the Readiness Assurance Test (individual and team) on the platform, which provides instant feedback to the student and the teacher. The tool allows the questionnaire to be carried out in person or online and to obtain statistics in real time. The platform supports the entire TBL cycle from a pedagogical point of view, allowing clarifications to be requested, supporting the appeals process and allowing peer evaluation in each team.

The course is carried out in hybrid and flexible mode. This means that the student can carry out the activity in person or online. Although online participation is not as effective (particularly for teamwork), the hybrid modality allows us to adapt to student's restrictions that may vary over time. Project work is done asynchronously and requires face-to-face oral defense.

VI. PEDAGOGICAL EVALUATION

The SE Department oversees teaching of 11 courses as part of the mandatory curricula of the Engineering and Systems Bachelor's degrees. In all courses, active learning strategies are applied to varying degrees, although Agile Software Engineering 2 can be considered a special case due to the simultaneous use of TBL and PBL with a software maintenance project and the use of technological platforms to support teaching advanced SE and DevOps concepts.

The evaluation of the classroom experience from a pedagogical point of view has been carried out considering two perspectives: teachers and students. For the evaluation from the point of view of the teachers, a series of semi-structured interviews and qualitative analysis were carried out. Teachers who applied the TBL methodology in other SE courses in the area were also included. To collect the students' perspective, the end-of-semester surveys carried out in the last three semesters were quantitatively and qualitatively analyzed.

A. Quantitative analysis of student perspective

For the quantitative analysis of the student perspective, we include in Table II the average satisfaction with the teacher collected by the student surveys. 11 sections are presented in 5 academic semesters during the years 2022 to 2024. Satisfaction with the teacher is measured on an ordinal scale

from 0 to 6, with a mean of 4.4 among all sections. The quantitative results show a general level of satisfaction, as well as exceptions related to the teacher assigned to the section.

TABLE II. STUDENT SATISFACTION

Semester	Section	Teacher	Average	Answers	Students
2024 S1	1	A	5,11	18	22
2024 S1	2	A	5,24	25	27
2024 S1	3	A	4,96	23	31
2023 S2	1	B	2,57	30	33
2023 S1	1	A	4,74	23	26
2023 S1	2	B	3,83	6	6
2023 S1	3	B	4	16	17
2022 S2	1	B	2,67	24	26
2022 S1	1	B	4,44	18	23
2022 S1	2	A	5,1	20	21
2022 S1	3	A	6	3	4

B. Qualitative analysis of student feedback

For the qualitative analysis of the class experience from the student perspective, we made a synthesis of the feedback received in the last academic semester (2024 S1). We separated the comments referring to the teacher assigned to the section, from the comments on the pedagogical methodology and the instructional design of the course. To carry out the qualitative analysis, we grouped these comments into three thematic categories: challenges of the TBL methodology, previous study effort and class exercises, improvements in the project and intermediate feedback. Examples of comments for each category are presented in Table III.

TABLE III. STUDENT FEEDBACK

Thematic category	Comments
TBL pedagogy challenges	<i>Working with a different team than the project, on topics not yet seen in class, with few instances of formal class</i> <i>The evaluation instance is for many the first contact we have where we apply what we have been reading</i> <i>very little material is offered outside of sources, there are no PPTs or abstracts, and there are almost no past evaluations to use</i> <i>The experience with InteDashboard was not the best</i>
Effort of previous study and exercises in class	<i>TBL methodology was very demanding for me reading 100 pages of the book for each RAT</i> <i>The course has too many evaluations</i> <i>I didn't like TBL's format, it takes up a lot of time and doesn't add up too much</i> <i>It was very annoying that all the classes were almost the same, separating into groups to end up writing some questions in a document and making commits</i> <i>I find the requirement of being present for what is basically almost all classes extremely demanding</i> <i>Requires the presence in class to perform the TBL exercises, they could perfectly be performed asynchronously</i>

² <https://www.intedashboard.com/>

Project improvements and intermediate feedback	<i>Five deliveries of the project Feedback on the intermediate instances of the deliveries of the project, especially after the 3rd, which is the largest The idea of intermediate deliveries was to receive early feedback; however, this did not happen</i>
--	---

C. Qualitative analysis of teacher perspective

The semi-structured interviews conducted with the teachers consisted of four sections: teaching experience, perception of the student experience, challenges and lessons learned. The interviews were analyzed qualitatively using open coding to identify recurring themes. We present the main insights extracted for each section in Table IV.

TABLE IV. TEACHERS INTERVIEW THEMES

Area	Recurring themes
Teaching experience	TBL implies a fundamental change in the teacher's role, stops exposing information and becomes a facilitator High effort of initial course preparation, grading effort for each unit still present Ongoing course maintenance effort outside the classroom
Perception of the student learning experience	The students' first contact with the new dynamic generates doubts and difficulties in applying it Complaints about the volume of preparation study material Increased participation and learning opportunities A gap in the learning experience is observed between students who participate in person and online
Challenges	Difficulty in constructing interesting application exercises and linking them to projects Operational aspects: group assembly, question bank, dynamics in hybrid mode, balancing the teacher-student effort Obstacles for hybrid teamwork Assess meaningful learning
Lessons learned	Active learning dynamics enrich courses Students do not value activities that do not have grading points in the same way Rethinking of the traditional lecture to introduce the thematic units and give the students an initial approach How to connect SE technology to theoretical content

Since this course applies the TBL methodology from the beginning, it is not possible to carry out a comparative study. The quantitative results show a medium-high level of satisfaction, similar to that of other subjects in the department. Our hypothesis is that the main correlated element to the evaluation is the teacher assigned to the group. From a qualitative point of view, the comments collected in the survey are mainly critical of the flipped classroom dynamics: the volume of study material and the lack of lessons taught by the teacher.

A lesson learned during the courses is that students are exposed to a context of courses where the use of active learning methodologies and teamwork as the main proposal is unusual. To mitigate this pedagogical transition, throughout the course cycles, mini lectures have been incorporated in some of the thematic units, practical support classes in technology and a greater focus on application exercises over preparation questionnaires.

VII. CONCLUSIONS

Active learning methodologies are key in engineering education. In the specific area of SE, there is a growing need

for deep learning of skills demanded by industry in an up-to-date technological context.

This article presents a classroom experience about combining TBL and PBL active learning strategies in an advanced SE course. The specific learning objectives of advanced agile and DevOps practices, the pedagogical methodologies to reach these objectives and the digital platforms applied for the realization of the course have been detailed.

We have carried out a pedagogical evaluation in a transversal way, considering both teacher's and student's perspectives. The evaluation shows that challenges persist, such as the effort of preparation and maintenance of the course for the teacher. Critical comments from students have been observed about the flipped classroom dynamics and the amount of preparation study material, which suggests that it is important to explain the learning objectives, the course methodology and the role of the teacher in that context more explicitly.

We have observed the increased engagement produced by active learning dynamics and their impact on the learning of skills, particularly in combining SE technical and soft skills. These pedagogical strategies allow the creation of a continuous and collaborative learning environment, where the teacher and students are an active part in the construction of knowledge and skills demanded for high-performance SE teams in a changing context.

REFERENCES

- [1] R. Hoda, N. Salleh, and J. Grundy, "The Rise and Evolution of Agile Software Development," *IEEE Softw.*, vol. 35, no. 5, pp. 58–63, Sep. 2018, doi: 10.1109/MS.2018.290111318.
- [2] I. A. Kautsar, M. R. Maika, A. N. Budiman, A. B. Setyawan, and J. Y. Awali, "Microservice Based Architecture: The Development of Rapid Prototyping Supportive Tools for Project Based Learning," in *2023 IEEE World Engineering Education Conference (EDUNINE)*, Mar. 2023, pp. 1–6. doi: 10.1109/EDUNINE57531.2023.10102884.
- [3] D. Akdur, "Analysis of Software Engineering Skills Gap in the Industry," *ACM Trans Comput Educ.*, vol. 23, no. 1, p. 16:1-16:28, Dec. 2022, doi: 10.1145/3567837.
- [4] G. Matturro, F. Raschetti, and C. Fontán, "A Systematic Mapping Study on Soft Skills in Software Engineering," *J. Univers. Comput. Sci.*, vol. 25, no. 1, pp. 16–41, 2019.
- [5] G. G. Borges and R. C. Grato de Souza, "Skills development for software engineers: Systematic literature review," *Inf. Softw. Technol.*, vol. 168, p. 107395, Apr. 2024, doi: 10.1016/j.infsof.2023.107395.
- [6] G. Van Helden, B. T. C. Zandbergen, M. M. Specht, and E. K. A. Gill, "Collaborative Learning in Engineering Design Education: A Systematic Literature Review," *IEEE Trans. Educ.*, vol. 66, no. 5, pp. 509–521, Oct. 2023, doi: 10.1109/TE.2023.3283609.
- [7] I. H. J. Song and C. Paczek, "Work in progress: Agile Methodologies for Online Software Engineering Education under the Pandemic," in *2023 IEEE World Engineering Education Conference (EDUNINE)*, Mar. 2023, pp. 1–4. doi: 10.1109/EDUNINE57531.2023.10102893.
- [8] S. Vasilache, "Offline and Online Active Learning: Lessons in Teaching Software Engineering to Multicultural Groups," presented at the International Conference on Software Engineering Advances, 2022.
- [9] M. Hernández-de-Menéndez, A. Vallejo Guevara, J. C. Tudón Martínez, D. Hernández Alcántara, and R. Morales-Menendez, "Active learning in engineering education. A review of fundamentals, best practices and experiences," *Int. J. Interact. Des. Manuf. IIJDeM.*, vol. 13, no. 3, pp. 909–922, Sep. 2019, doi: 10.1007/s12008-019-00557-8.
- [10] J. I. Irabedra, G. Mousqués, and M. Solari, "Active learning in software engineering: A rapid review," presented at the 50a Conferencia Latinoamericana de Informática (L CLEI 2024), Bahia Blanca, Argentina.

QP3
QP4

- [11] D.-M. Córdova-Esparza, J.-A. Romero-González, K.-E. Córdova-Esparza, J. Terven, and R.-E. López-Martínez, "Active Learning Strategies in Computer Science Education: A Systematic Review," *Multimodal Technol. Interact.*, vol. 8, no. 6, Art. no. 6, Jun. 2024, doi: 10.3390/mti8060050.
- [12] J. P. Cambrónero, T. H. Y. Dang, N. Vasilakis, J. Shen, J. Wu, and M. C. Rinard, "Active learning for software engineering," in *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, in Onward! 2019. New York, NY, USA: Association for Computing Machinery, Oct. 2019, pp. 62–78. doi: 10.1145/3359591.3359732.
- [13] B. R. Maxim, A. Decker, and J. J. Yackley, "Student Engagement in Active Learning Software Engineering Courses," in *2019 IEEE Frontiers in Education Conference (FIE)*, Oct. 2019, pp. 1–5. doi: 10.1109/FIE43999.2019.9028644.
- [14] M. Marques, S. F. Ochoa, M. C. Bastarrica, and F. J. Gutierrez, "Enhancing the Student Learning Experience in Software Engineering Project Courses," *IEEE Trans. Educ.*, vol. 61, no. 1, pp. 63–73, Feb. 2018, doi: 10.1109/TE.2017.2742989.
- [15] B. Pérez and Á. L. Rubio, "A Project-Based Learning Approach for Enhancing Learning Skills and Motivation in Software Engineering," in *Proceedings of the 51st ACM Technical Symposium on Computer Science Education*, in SIGCSE '20. New York, NY, USA: Association for Computing Machinery, Feb. 2020, pp. 309–315. doi: 10.1145/3328778.3366891.
- [16] B. R. Maxim and J. J. Yackley, "Using Active Learning to Teach Software Engineering in Game Design Courses," in *Software Engineering for Games in Serious Contexts: Theories, Methods, Tools, and Experiences*, K. M. L. Cooper and A. Bucchiarone, Eds., Cham: Springer Nature Switzerland, 2023, pp. 189–213. doi: 10.1007/978-3-031-33338-5_9.
- [17] M. R. de A. Souza, L. Veado, R. T. Moreira, E. Figueiredo, and H. Costa, "A systematic mapping study on game-related methods for software engineering education," *Inf. Softw. Technol.*, vol. 95, pp. 201–218, Mar. 2018, doi: 10.1016/j.infsof.2017.09.014.
- [18] M. Prince, "Does Active Learning Work? A Review of the Research," *J. Eng. Educ.*, vol. 93, no. 3, pp. 223–231, 2004, doi: 10.1002/j.2168-9830.2004.tb00809.x.
- [19] A. Adorjan and M. Nunez-Del-Prado, "Fostering 21 Century Learning and Innovation Competencies Through Students' Online Collaborative Activities in Software Engineering Courses," in *2018 IEEE World Engineering Education Conference (EDUNINE)*, Mar. 2018, pp. 1–4. doi: 10.1109/EDUNINE.2018.8450987.
- [20] A. Adorjan and M. Solari, "Software Engineering Project-Based Learning in an Up-To-Date Technological Context," in *2021 IEEE URUCON*, Nov. 2021, pp. 486–491. doi: 10.1109/URUCON53396.2021.9647348.
- [21] L. Anderson et al., *Taxonomy for Learning, Teaching, and Assessing, A: A Revision of Bloom's Taxonomy of Educational Objectives*. New York: Pearson, 2000.
- [22] L. K. Michaelsen and M. Sweet, "The essential elements of team-based learning," *New Dir. Teach. Learn.*, vol. 2008, no. 116, pp. 7–27, 2008, doi: 10.1002/tl.330.
- [23] I. Cabrera, J. Villalon, and J. Chavez, "Blending Communities and Team-Based Learning in a Programming Course," *IEEE Trans. Educ.*, vol. 60, no. 4, pp. 288–295, Nov. 2017, doi: 10.1109/TE.2017.2698467.
- [24] R. Ramos-Rodriguez, M. Calle, G. Coronell, and J. E. C. Becerra, "Transdisciplinarity and Team-Based Learning: Strategies for an Introductory Programming Course," *IEEE Trans. Educ.*, vol. 67, no. 2, pp. 317–326, Apr. 2024, doi: 10.1109/TE.2024.3367617.
- [25] H. Kondo and A. Hazeyama, "Systematic Literature Review on Educational Effectiveness of Project-Based Learning for Software Development," in *2022 29th Asia-Pacific Software Engineering Conference (APSEC)*, Dec. 2022, pp. 584–585. doi: 10.1109/APSEC57359.2022.00092.
- [26] A. Espinoza-Robles, J. Trujillo-Trejo, A. Fermín-Pérez, J. Chávez-Soto, V. Bustamante-Olivera, and N. Muñoz-Casildo, "Application of Project-Based Learning to Improve the Teaching and Learning Outcomes of a Course in Information System Design in a Computer Science Department," in *2022 IEEE World Engineering Education Conference (EDUNINE)*, Mar. 2022, pp. 1–4. doi: 10.1109/EDUNINE53672.2022.9782317.
- [27] J. A. Hurtado, A. C. Useche, and B. S. Masiero, "Project-Based Learning: Authentic Engineering Assessment Supported by Model Design," *Int. J. Eng. Pedagogy IJEP*, vol. 13, no. 6, Art. no. 6, Sep. 2023, doi: 10.3991/ijep.v13i6.38539.
- [28] M. A. Nelson and L. Ponciano, "Experiences and insights from using Github Classroom to support Project-Based Courses," in *2021 Third International Workshop on Software Engineering Education for the Next Generation (SEENG)*, May 2021, pp. 31–35. doi: 10.1109/SEENG53126.2021.00013.
- [29] B. O'Dwyer, "All-in-One Team-Based Learning (TBL) Technology: Profiling the InteDashboard Technology Platform," in *Perspectives on Digitally-Mediated Team Learning*, L. O. Campbell, R. Hartshorne, and R. F. DeMara, Eds., Cham: Springer International Publishing, 2021, pp. 95–118. doi: 10.1007/978-3-030-77614-5_6.